Hands-On-Learning: Secure Financial Platform: Implement BankTech's Compliance Framework

Learning Objectives

By the end of this activity, learners will be able to:

- LO1: Configure OAuth 2.0 authentication and IAM authorization for LLM API endpoints.
- LO2: Implement rate limiting using API Gateway to prevent abuse and cost attacks.
- LO3: Build prompt injection defense systems using input validation and sanitization.



- LO4: Deploy output filtering to prevent PII leakage in model responses.
- LO5: Establish audit logging that meets GDPR, SOC 2, and HIPAA compliance requirements.



Description

In this hands-on practice, you will implement a comprehensive security framework for BankTech's LLM-powered financial advisor, addressing API protection, prompt injection defense, and compliance-ready audit logging to pass regulatory audits.

Structure of the Activity

Title: Hands-on Practice: Implementing Security and Compliance Framework for LLM Financial Platform

Tools to be used:

- API Gateway (Kong or AWS API Gateway)
- OWASP ZAP for security testing
- ELK Stack (Elasticsearch, Logstash, Kibana) for logging
- HashiCorp Vault for secrets management
- Python for middleware implementation

coursera

Setup Requirements:

- Docker for running Kong API Gateway locally.
- Python 3.9+ with pip for middleware development.
- OWASP ZAP installed for penetration testing.
- ELK Stack (can use Docker Compose for local setup).
- At least 8GB RAM for running security tools.

Scenario

You are a Security Engineer at BankTech, a global bank launching an LLM-powered financial advisor for 5 million customers. A penetration test exposed critical vulnerabilities: unsecured API endpoints, prompt injection susceptibility, and inadequate audit logging. You have 45 days to implement a security framework that passes regulatory audits.

Getting Started:

- Download and extract LLM-Deployment-Starter-Files.zip from the course resources.
- Navigate to the Module2_Security folder and review M2_api_gateway_config.yaml and M2_injection_tests.json.
- Review M2_compliance_matrix.csv to understand SOC2/GDPR/PCI-DSS requirements.
- Build security controls incrementally using the provided templates, testing with M2_injection_tests.json.

Activity 1: Implement API Security Controls

Aligned LOs: LO1, LO2

Steps to Complete:

Step 1: Configure JWT Authentication

- Set up JWT authorizer in API Gateway.
- Configure token validation for issuer, audience, and expiration.
- Test with valid and invalid tokens to verify rejection.

Step 2: Create IAM Policies

- Define a least-privilege IAM role for the inference server.
- Grant only S3 read access for model weights and CloudWatch write for logs.
- Create a separate role for application accessing the LLM endpoint.

Step 3: Implement Rate Limiting

- Configure per-user rate limits (1,000 requests/hour for basic tier).
- Set up tiered limits (5,000 for professional, 20,000 for enterprise).
- Add response headers indicating remaining quota.

Step 4: Test Security Controls

- Verify unauthenticated requests are rejected with 401.
- Confirm rate limiting returns 429 when exceeded.
- Document test results for compliance evidence.

Tips:

- Use short token expiration for security.
- Log all authentication failures for security monitoring.
- Consider burst limits for handling legitimate traffic spikes.

Activity 2: Build Prompt Injection Defenses with Output Filtering

Aligned LOs: LO3, LO4

Steps to Complete:

Step 1: Implement Input Validation

- Create Python middleware for request validation.
- Add regex patterns to detect common injection signatures.
- Block prompts containing "ignore previous instructions" and similar patterns.

Step 2: Add Prompt Sanitization

- Strip special characters and escape sequences.
- Implement length limits (max 4096 tokens).
- Add system prompt protection to prevent override attempts.

Step 3: Configure Output Filtering for PII

- Implement PII detection using regex patterns for SSN, credit cards, etc.
- Add financial data redaction for account numbers and balances.
- Configure response blocking when PII leakage is detected.

coursera

Step 4: Test Defense Mechanisms

- Test against provided injection_tests.json attack patterns.
- Verify all injection attempts are blocked and logged.
- Confirm PII filtering catches sensitive data in responses.
- Document test results for compliance evidence.

coursera

Tips:

- Layer multiple defense mechanisms—no single control is sufficient.
- Log blocked requests for security analysis without storing malicious content.
- Consider using NeMo Guardrails for production deployments.

Activity 3: Implement Compliance-Ready Audit Logging

Aligned LO: LO5

Steps to Complete:

Step 1: Design Log Schema

- Define structured log format with timestamp, user_id, and session_id.
- Include prompt hash (not full content), response status, and token count.
- Add source IP, user agent, and request duration fields.

Step 2: Configure ELK Stack

- Deploy Elasticsearch, Logstash, and Kibana using Docker Compose.
- Configure Logstash pipeline to parse and index LLM logs.
- Set up log retention policies (7 years for HIPAA, 6 years for SOC 2).

Step 3: Implement Compliance Controls

- Add GDPR right-to-deletion support with pseudonymization.
- Configure role-based access control for log viewing.
- Enable log encryption at rest and in transit.

Step 4: Create Compliance Documentation

- Map logging controls to SOC 2, GDPR, and HIPAA requirements.
- Create sample compliance queries for auditors.
- Document data retention and deletion procedures.

Tips:

- Never log full prompts—use hashes for compliance while protecting IP.
- Consider separate logging for security events vs. operational metrics.
- Set up alerts for anomalous access patterns.

Compile Your Findings

coursera

Requirement 1: API Authentication Configuration

Demonstrate OAuth 2.0 and IAM authorization for LLM endpoints.

Include:

- API Gateway configuration with JWT authentication.
- IAM policy documents with least-privilege permissions.
- Evidence of rejected unauthorized requests (401 responses).
- Token validation test results.

Requirement 2: Rate Limiting Implementation

Demonstrate abuse prevention through tiered rate limiting.

Include:

- Rate limiting configuration for three tiers.
- Evidence of 429 responses when limits exceeded.
- Response headers showing quota status.
- Burst limit configuration for traffic spikes.

Requirement 3: Prompt Injection Defense

Demonstrate layered defense against LLM-specific attacks.

Include:

- Input validation middleware code.
- Prompt sanitization logic with regex patterns.
- Test results showing blocked injection attempts.
- Security event logging configuration.

Requirement 4: PII Output Filtering

Demonstrate data leakage prevention in model responses.

Include:

- Output filtering configuration for PII patterns.
- Financial data redaction rules.
- Test results showing PII detection and blocking.
- Response redaction examples.

Requirement 5: Compliance-Ready Audit Logging

Demonstrate regulatory compliance capabilities.

Include:

- Log schema and ELK configuration.
- Compliance matrix mapping controls to SOC 2, GDPR, HIPAA.
- Sample audit queries for compliance verification.
- Business analysis (3-5 sentences): Were all vulnerabilities remediated? Is the system ready for a SOC 2 audit? What additional controls might be needed?

Summary

This activity provides hands-on experience with enterprise LLM security through practical implementation of layered defense mechanisms. Participants have developed competency in API authentication, prompt injection defense, and compliance-ready audit logging, all of which are directly applicable to regulated industries such as financial services and healthcare.

Key takeaways:

- LLM security requires layered defenses—authentication, input validation, output filtering, and audit logging work together to create comprehensive protection.
- Prompt injection attacks are unique to LLM applications and require specialized defenses beyond traditional API security, as highlighted in OWASP Top 10 for LLMs.
- Compliance-ready audit logging with proper retention policies and access controls is essential for regulated industries—a lesson learned from the 1,100+ exposed Ollama servers discovered by Cisco Talos.

Resources

Primary Materials (from LLM-Deployment Repository files):

- M2_api_gateway_config.yaml - Kong API Gateway configuration with rate limiting stubs
- M2_injection_tests.json - 20+ prompt injection test cases across categories
- M2_prompt_validator.py - Input validation and PII detection module
- M2_audit_logger.py - Compliance-ready audit logging for SOC2/GDPR/PCI-DSS
- M2_compliance_matrix.csv - 20 controls mapped to frameworks with status tracking

Reference Documentation:

- OWASP Top 10 for LLM Applications: https://genai.owasp.org/llm-top-10/
- Kong API Gateway Documentation: https://docs.konghq.com/
- ELK Stack Documentation: https://www.elastic.co/guide/

Solution/Exemplar:

- Hands_On_Practice_Module2_Solution

Share The Project

Document: Save the report in PDF format.

Upload: Share in the “Peer Review” area of the course shell with a brief description.

Instructions for Documenting and Sharing The Project for Peer Review

1. Document the Project

- Use word processing software to create your report.
- Ensure all sections of the project document structure are completed.
- Proofread and edit your report for clarity and accuracy.

2. Share the Project

- Save your report in PDF format.
- Upload your documents to the “Peer Review” area in the course shell.
- Provide a brief description of your project in the submission post.

Instructions for Documenting and Sharing The Project for Peer Review

3. Peer Review

- Review the projects submitted by your peers.
- Provide constructive feedback and comments on their work.